



Experiment – 5

Name: Simranjeet Kaur

UID: 24BAI70048

Branch: CSE – AIML (CS221)

Section/Group: 24AIT_NTPP-2A

Semester: 5th

Date of Performance: /08/2026

Subject: Full Stack-II

Subject Code: 24CSP-337

EXPERIMENT – 5.1

1. **AIM:** To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture.

2. **OBJECTIVE:**

1. To understand REST API design principles
2. To implement CRUD APIs using Spring Boot
3. To enforce consistent request-response structures
4. To apply validation using Bean Validation

3. **SOFTWARE REQUIREMENTS:**

1. Java (JDK 17+)
2. Spring Boot
3. IntelliJ IDEA
4. Postman / API testing tool
5. Maven
6. Spring Web, Spring Data JPA and Bean Validation

4. **THEORY**

REST (Representational State Transfer) is an architectural style used for designing scalable and loosely coupled web services. In a RESTful system, data is treated as resources and accessed through standard HTTP methods such as GET, POST, PUT and DELETE. The Spring Boot application developed for this experiment follows a layered architecture so that request handling, business logic and data access remain separated.

- Controller Layer → Handles HTTP requests and maps REST endpoints.
- Service Layer → Contains application and business logic.
- Repository Layer → Performs database interaction using Spring Data JPA.

The API follows stateless communication and a consistent URI structure. Bean Validation is applied to incoming request data using annotations such as `@NotBlank`, `@NotNull` and `@Size`. Invalid data is rejected before it reaches the business logic. A common response structure is used so that the frontend receives predictable success and error responses.

CORS (Cross-Origin Resource Sharing) is configured to permit controlled communication between a frontend application and the Spring Boot backend when they run on different origins..

5. **IMPLEMENTATION/PROCEDURE**

1. Create a Spring Boot project in IntelliJ IDEA with the required dependencies.
2. Create the Controller, Service, Repository, Model and DTO/response classes.
3. Implement CRUD REST endpoints for the post-scheduling application.
4. Use HTTP GET, POST, PUT and DELETE methods for resource operations.
5. Apply Bean Validation annotations to incoming request fields.

6. Create a common response structure for successful API operations.
7. Configure CORS so the frontend can communicate with the backend.
8. Run the Spring Boot application and test the endpoints using Postman.
9. Verify valid requests, invalid requests and CRUD operations.

6. CODE

```
package com.example.restapi;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class RestapiApplication {

    public static void main(String[] args) {
        SpringApplication.run(RestapiApplication.class, args);
    }
}
```

7. OUTPUT

The output is a Spring Boot REST API for managing posts and scheduling data. The API provides CRUD endpoints that can be tested through Postman. Valid requests return a consistent response structure, while invalid request data is rejected through Bean Validation. The configured CORS policy allows the frontend application to communicate with the backend.

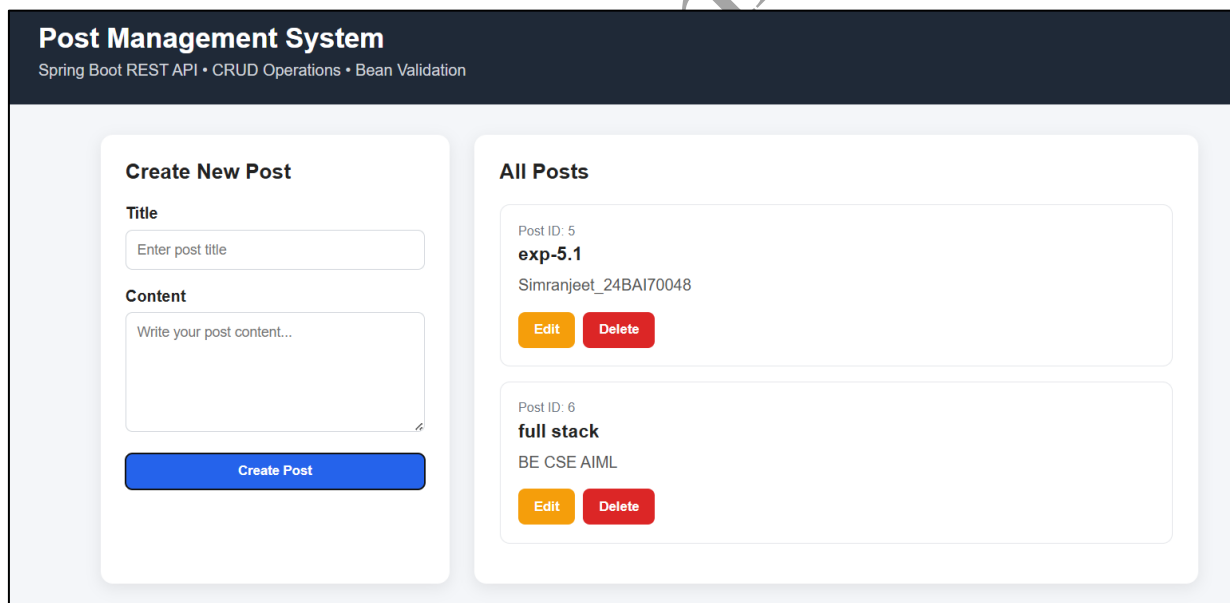


Fig 5.1: Spring Boot REST API tested using Postman

8. LEARNING OUTCOME

- Understand the design and implementation of RESTful APIs using Spring Boot.
- Implement CRUD operations using standard HTTP methods.
- Apply Bean Validation to incoming API requests.
- Design consistent success response structures for frontend integration.
- Understand layered architecture using Controller, Service and Repository components.
- Configure CORS for frontend-backend communication.
- Test REST endpoints using an API testing tool such as Postman.

EXPERIMENT –5.2

1. **AIM:** To implement global exception handling and structured logging for building robust and observable backend systems.

2. **OBJECTIVE:**

- To handle exceptions centrally using `@ControllerAdvice`
- To implement logging mechanisms for request tracking
- To use correlation IDs for tracing requests
- To improve system observability and debugging

3. **SOFTWARE REQUIREMENTS:**

1. Java (JDK 17+)
2. Spring Boot
3. IntelliJ IDEA
4. Postman / API testing tool
5. SLF4J / Logback logging framework

4. **THEORY**

In a backend application, handling exceptions separately inside every controller can result in repeated code and inconsistent error messages. Spring Boot provides global exception handling through `@ControllerAdvice` or `@RestControllerAdvice`. A centralized exception handler receives application exceptions and converts them into a uniform error response. Code maintainability is improved because exception-handling logic is kept in one place. Error responses remain consistent across different REST endpoints. Debugging becomes easier because failures can be logged centrally. Logging is used to record important events such as incoming requests, request paths, execution details and exceptions. Filters and interceptors can act as middleware around requests and are useful for logging and request tracing. A correlation ID can be assigned to every incoming request and stored in MDC (Mapped Diagnostic Context). The same identifier can then appear in log messages, allowing developers to trace the lifecycle of an individual request, especially when multiple requests are being processed.

5. **IMPLEMENTATION/PROCEDURE**

1. Create custom exception classes for application-specific errors.
2. Create a global exception handler using `@RestControllerAdvice`.
3. Handle resource-not-found and validation-related exceptions centrally.
4. Return a common error response containing status, message and request information.
5. Create a request logging filter to intercept incoming HTTP requests.
6. Generate or read a correlation ID for each request.
7. Store the correlation ID in MDC so it can be included in log messages.
8. Log request information and exceptions using SLF4J/Logback.
9. Run the application and generate both successful and erroneous requests through Postman.
10. Verify that the response contains a meaningful error message and that the corresponding request can be traced in the logs.

6. **CODE**

```
package com.example.restapi;
```

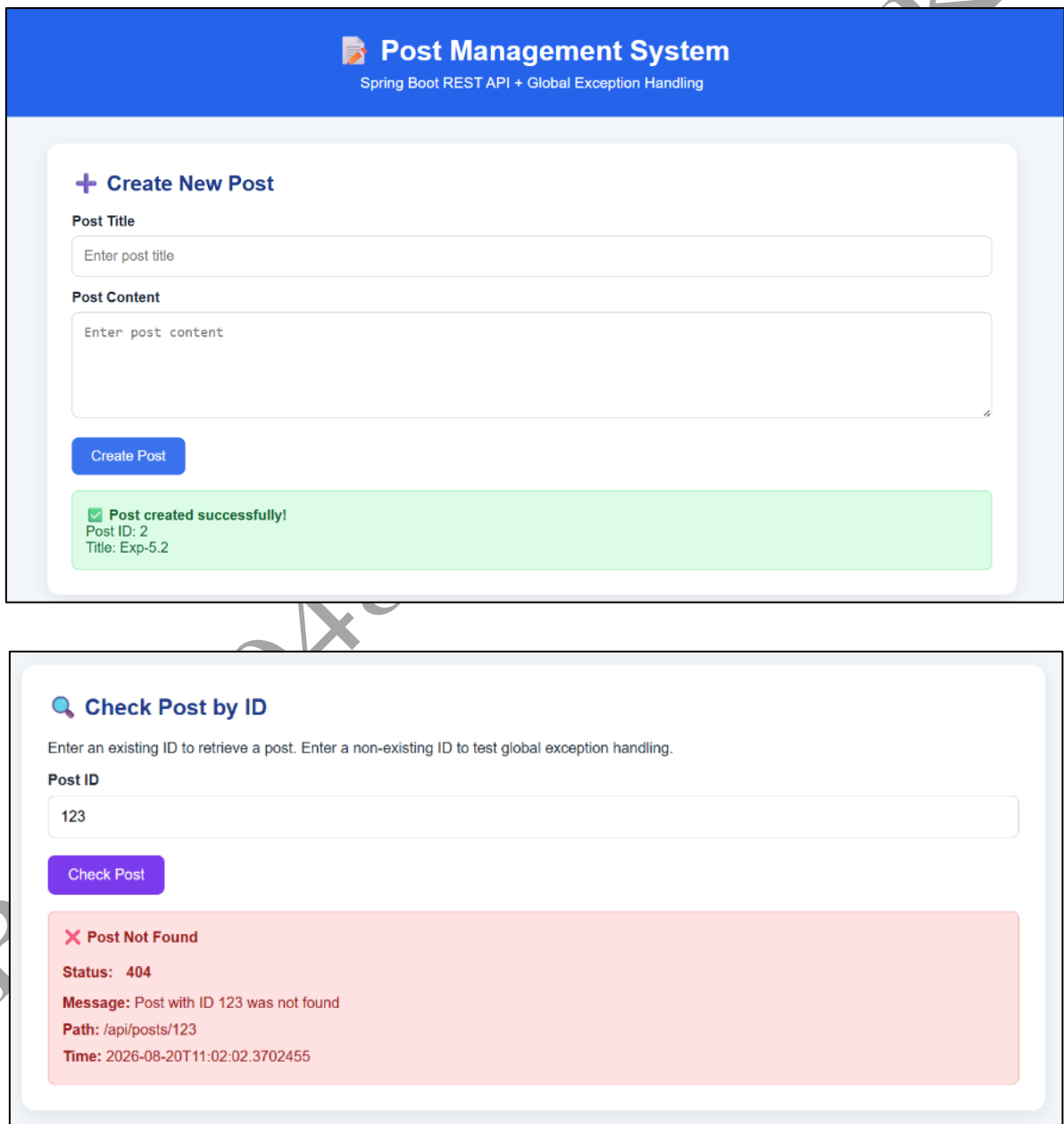
```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication  
public class RestapiApplication {
```

```
public static void main(String[] args) {  
    SpringApplication.run(RestapiApplication.class, args);  
}
```

7. OUTPUT

The output demonstrates centralized exception handling and request logging in the Spring Boot REST API. When an invalid or unavailable resource is requested, the application returns a structured error response instead of an unhandled exception. The request is also recorded in the application logs with a correlation ID, HTTP method, URI, response status and execution time. This makes the backend easier to monitor, debug and maintain.



The image displays two screenshots of a web application titled "Post Management System" with the subtitle "Spring Boot REST API + Global Exception Handling".

The top screenshot shows the "Create New Post" form. It has two input fields: "Post Title" (containing "Enter post title") and "Post Content" (containing "Enter post content"). Below the fields is a blue "Create Post" button. A green success message box at the bottom states: "✓ Post created successfully! Post ID: 2 Title: Exp-5.2".

The bottom screenshot shows the "Check Post by ID" form. It has a single input field for "Post ID" containing the value "123". Below the field is a purple "Check Post" button. A red error message box at the bottom displays the following structured response: "✗ Post Not Found", "Status: 404", "Message: Post with ID 123 was not found", "Path: /api/posts/123", and "Time: 2026-08-20T11:02:02.3702455".

Fig 5.2: Global exception handling and structured request logging in Spring Boot

8. LEARNING OUTCOME

- Understand the need for centralized exception handling in REST applications.
- Implement `@RestControllerAdvice` and `@ExceptionHandler` for uniform error processing.
- Create and use custom exceptions for application-specific failures.
- Use filters for request interception and logging.
- Understand MDC and correlation IDs for request tracing.
- Improve debugging and observability through structured logs.
- Build a more robust and maintainable Spring Boot backend.

24BAI70048 — Simranjeet Kaur